LIÈGE université
Sciences Appliquées

INFO0902-1: Structures de Données et Algorithmes

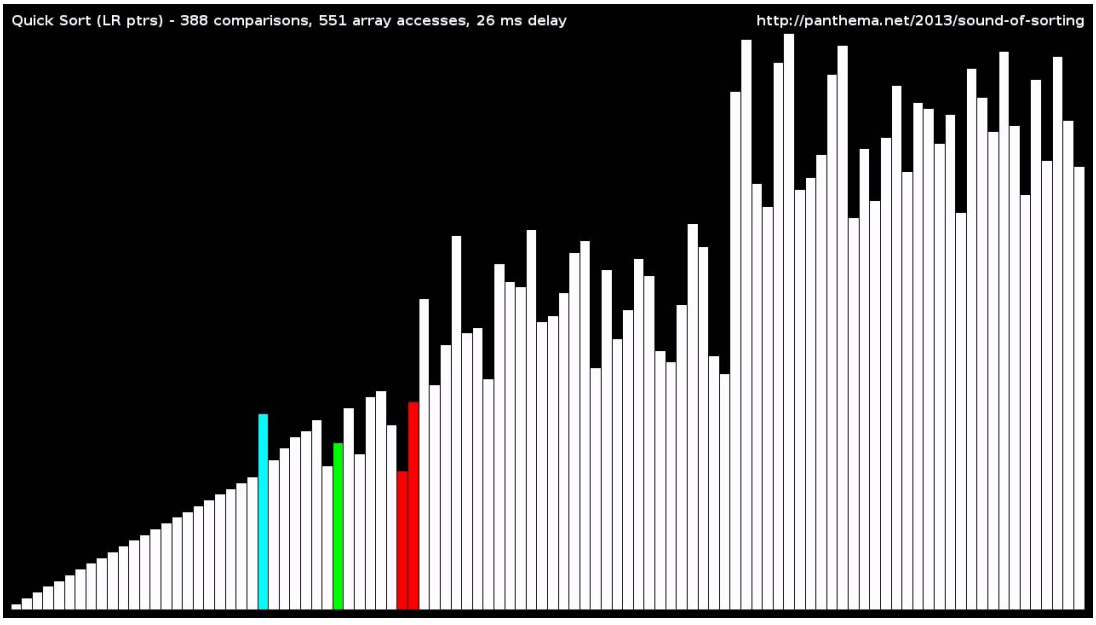
Université de Liège - Faculté des sciences appliquées

Student: Alexandre Detienne

Student-ID: s2301654

Project 1: Autocompletion

Report



Année Académique 2025-2026

Contents

1 Temps de calcul des algorithmes de tri: 2

2 Discussion des résultats sur les tris: 2

3 Analyse théorique des trois approches: 2

3.0.1 Analyse de complexités de MergeSort: . . . 2
3.0.2 Analyse de complexité de QuickSort: . . . 3
3.0.3 Analyse de complexité de `BinarySearch` . . . 4
3.1 Analyse de AutoCompleteLinear: . . . 4
3.2 Analyse de complexité de AutoCompleteSort: . . . 4
3.2.1 Analyse de complexité de `acCreate` . . . 4
3.2.2 Analyse de complexité de `acComplete` . . . 4
3.3 Analyse de complexité de AutoCompletePQ.c: . . . 5
3.3.1 Analyse de complexité de `acCreate` . . . 5
3.3.2 Analyse de complexité de acComplete: . . . 5
3.4 Résumé de l'analyse des complexités des 3 algorithmes d'auto-complétions: . . . 6

4 Comparaison empirique des trois approches d'auto-complétion: 6

5 Discussion des résultats expérimentaux 7
5.1 AutoCompleteLinear . . . 7
5.2 AutoCompleteSort . . . 8
5.3 AutoCompletePQ . . . 8

6 Conclusion 8

7 Remarques: 8

1 Temps de calcul des algorithmes de tri:

Nous avons mesuré les temps de calculs des algorithmes QuickSort et MergeSort, sur les fichiers: `actors_25k.txt`, `actors_250k.txt` et `actors_2500k.txt`. Voici les résultats dans le tableau ci-dessous:

Fichier	**MergeSort (lexico)**	**MergeSort (poids)**	**QuickSort (lexico)**	**QuickSort (poids)**
actors_25k.txt	0.009642	0.003327	0.009146	1.155062
actors_250k.txt	0.062467	0.016463	0.046392	115.357325
actors_2500k.txt	0.057960	0.016942	0.049492	114.956062

Table 1: Temps de calcul en secondes des algorithmes de tri

2 Discussion des résultats sur les tris:

D'un point de vue théorique, nous avons vu que l'algorithme `QuickSort` a:

- une complexité en temps de l'ordre de $\Theta(n \log n)$ dans le meilleur cas.
- une complexité en temps de l'ordre de $\Theta(n^2)$ dans le pire cas.

Le pire cas de `QuickSort` se produit lorsque le pivot reste systématiquement à une extrémité du tableau après chaque partition, c'est à dire lorsqu'il est toujours le plus petit ou le plus grand élément du tableau. Cela entraîne des partitions déséquilibrées, puisqu'on aura un tableau vide et un tableau de taille n-1. Cette situation apparaît lorsque le tableau est trié(croissant ou décroissant) et que le pivot est mal choisi(le dernier ou le premier élément).

Le meilleur cas de `QuickSort` se produit lorsque le pivot est toujours proche de la médiane du tableau. Cela entraîne des partitions plus ou moins équilibrées, puisqu'on aura deux sous-tableaux qui seront approximativement de la même taille(n/2).

Contrairement au `QuicksSort`, `MergeSort` présente une complexité en temps de $\Theta(n \log n)$ dans tous les cas, car le tableau est toujours divisé en deux parties équilibrées. Ses performances ne dépendent pas des données initiales.

Les complexités théoriques sont cohérentes avec nos résultats. Pour le tri lexicographique, l'efficacité des deux algorithmes est presque identique. Pour le tri par poids, `QuickSort` se retrouve dans un cas proche du pire cas puisque les données sont déjà triées par ordre décroissant de poids. `MergeSort` reste plus efficace.

L'algorithme le plus adapté pour ce problème est `MergeSort`. En effet, cet algorithme ne dépend pas des données et garantit une complexité en temps de $\Theta(n \log n)$ dans tous les cas. Même si `QuickSort` est un peu plus rapide pour le tri lexicographique, `MergeSort` est plus stable.

3 Analyse théorique des trois approches:

Dans cette section, nous allons analyser les complexités en temps et en espace des trois approches d'autocomplétion dans le meilleur et le pire cas, en fonction de:

- le nombre total de termes, N
- le nombre de termes dont la requête est un préfixe, m
- le nombre de termes à afficher, k
- le nombre de lettre dans la requête, q

Nous allons profiter de cette section pour calculer les complexités en temps et en mémoire des algorithmes qui sont utilisés plusieurs fois dans nos implémentations d'autocomplétion: `MergeSort`, `QuickSort` et `BinarySearch`.

3.0.1 Analyse de complexités de MergeSort:

On note $T(N)$ le temps d'exécution de l'algorithme pour un tableau de taille N.

Dans l'implémentation de l'algorithme, `mergeS` divise récursivement le tableau en deux sous-tableaux de taille $N/2$, puis appelle la fonction `merge` pour les fusionner. La fonction `merge` parcourt l'ensemble des éléments une seule fois, ce qui donne un coût en $\Theta(N)$.

Nous obtenons donc la récurrence suivante:

$$T(N) = 2 \times T\left(\frac{N}{2}\right) + \Theta(N)$$

que nous allons résoudre à l'aide de la méthode d'arbre de récursion. Parcourons quelques itérations ensemble:

- **Niveau 0:** $T(N)$
- **Niveau 1:** $T(N) = 2T\left(\frac{N}{2}\right) + N$
- **Niveau 2:** $T(N) = 4T\left(\frac{N}{4}\right) + 2N$
- **Niveau 3:** $T(N) = 8T\left(\frac{N}{8}\right) + 3N$
- $\vdots$
- **Niveau k:** $T(N) = 2^k T\left(\frac{N}{2^k}\right) + kN$

La récursion se termine lorsque:

$$T\left(\frac{N}{2^k}\right) = T(1) \Leftrightarrow \frac{N}{2^k} = 1 \Leftrightarrow k = \log_2(N)$$

En substituant ce résultat dans la récurrence au niveau k, nous obtenons:

$$T(N) = N \times T(1) + N \times \log_2(N) \in \Theta(N \log N)$$

`MergeSort` a une complexité en mémoire de $\Theta(N)$, car nous n'allouons pas de mémoire supplémentaire dans l'algorithme. `MergeSort` a une complexité en mémoire théorique de $\Theta(1)$, mais mon implémentation alloue des tableaux temporaires(3) de taille N. Dès lors, la complexité de mon algorithme est $\Theta(N)$. Il est possible d'implémenter l'algorithme en $O(1)$ avec inline swap[1] mais cette implémentation est très compliquée et ne serait pas le fruit de mon travail.

3.0.2 Analyse de complexité de QuickSort:

On note $T(N)$ le temps d'exécution de l'algorithme pour un tableau de taille N.

Dans cette implémentation, la fonction choisit toujours le dernier élément du tableau comme pivot, puis parcourt le reste du tableau une seule fois afin de partitionner les éléments en deux sous-tableaux. Le coût de cette opération est en $\Theta(N)$. Si le pivot se retrouve à la position i, les deux partitions du tableau N seront de taille i et $N - i - 1$:

$$T(N) = T(i) + T(N - i - 1) + \Theta(N)$$

Dans le meilleur cas, le tableau N sera toujours coupé parfaitement en deux. On se retrouve donc avec:

$$T(N) = 2T\left(\frac{N}{2}\right) + \Theta(N)$$

. Il s'agit de la même équation de récurrence que pour `MergeSort`. En conclusion, dans le meilleur cas, `QuickSort` est de l'ordre de $T(N) \in \Theta(N \log N)$.

Dans le pire cas, le pivot se trouvera toujours sur une extrémité du tableau. Dans cette situation, nous aurons un sous-tableau vide et un sous-tableau de taille $N - 1$, ce qui donne la récurrence suivante:

$$T(N) = T(N - 1) + \Theta(N)$$

Parcourons quelques itérations ensemble:

- $T(N) = T(N - 1) + cN$
- $T(N) = T(N - 2) + c(N - 1) + cN$
- $T(N) = T(N - 3) + c(N - 2) + c(N - 1) + cN$
- $\vdots$
- $T(N) = T(1) + c \cdot (2 + 3 + \cdots + N)$

En utilisant la formule de la série arithmétique, nous obtenons:

$$T(N) = T(1) + c \cdot \sum_{j=2}^{N} j = T(1) + c \cdot \left(\frac{N(N+1)}{2} - 1\right) \in \Theta(N^2)$$

`QuickSort` a une complexité en mémoire de $\Theta(1)$, car nous n'allouons pas de mémoire supplémentaire dans l'algorithme.

[1] `http://akira.ruc.dk/~keld/teaching/algoritmedesign_f04/Artikler/04/Huang88.pdf`

3.0.3 Analyse de complexité de `BinarySearch`

La recherche binaire est un algorithme itératif qui, à chaque itération, divise par deux le tableau. Au départ, le tableau contient N éléments. Après une itération, il en reste au plus $\frac{N}{2}$, puis $\frac{N}{4}$ après deux itérations, et plus généralement, après k itérations, il reste :

$$\frac{N}{2^k}.$$

L'algorithme s'arrête lorsque cette quantité vaut 1, ce qui donne :

$$\frac{N}{2^k} = 1 \quad \Leftrightarrow \quad 2^k = N \quad \Leftrightarrow \quad k = \log_2 N.$$

Le nombre d'itérations est donc $\log_2 N$. Cependant, à chaque itération, la fonction de comparaison `compareBSTerm` parcourt les $|q|$ caractères de la requête, ce qui lui confère un coût de $O(|q|)$ par appel. La complexité totale de `binarySearchLow` est donc :

$$T(N) \in O(|q| \cdot \log N)$$

3.1 Analyse de AutoCompleteLinear:

Principe : On parcourt la liste des N termes par ordre décroissant de poids et on sélectionne les k premiers termes dont la requête est un préfixe.

Complexité en temps :

- **Meilleur cas :** Les k premiers termes du tableau correspondent déjà au préfixe. On s'arrête après k comparaisons : $\Theta(k)$.
- **Pire cas :** Les k termes compatibles se trouvent en fin de tableau, ou bien $m < k$ termes correspondent au préfixe. On parcourt alors l'intégralité du tableau : $\Theta(N)$.

Complexité en espace :

- `acCreate` n'effectue aucune allocation de mémoire, sa complexité en mémoire est $\Theta(1)$.
- `acComplete` remplit un tableau de k pointeurs vers les termes sélectionnés, soit $\Theta(k)$.

3.2 Analyse de complexité de AutoCompleteSort:

Principe : La liste des N termes est d'abord triée par ordre lexicographique. Lors d'une requête, une recherche binaire permet d'identifier les m termes consécutifs dont la requête est un préfixe. Ces m termes sont ensuite triés par ordre décroissant de poids et les k premiers sont extraits et retournés.

3.2.1 Analyse de complexité de `acCreate`

Dans notre implémentation, la fonction `acCreate` trie les données par ordre lexicographique à l'aide d'un des deux algorithmes de tri `MergeSort` ou `QuickSort`. Les données étant dans un ordre quelconque, nous ne nous retrouvons pas dans le pire cas de `QuickSort`. Dès lors, la complexité en temps de cette fonction est de l'ordre de:

$$\Theta(N \log N)$$

peu importe l'algorithme utilisé.

La complexité en mémoire de `acCreate` est de $\Theta(N)$ puisqu'il y a une allocation temporaire de tableaux d'indices.

3.2.2 Analyse de complexité de `acComplete`

Avant de pouvoir calculer la complexité de `acComplete`, nous devons d'abord analyser la fonction `commonPrefix`. Celle-ci parcourt les m termes du tableau à partir de l'indice `idx` tant qu'ils partagent le même préfixe que la requête. À chaque itération, elle appelle `strncmp` qui compare les deux chaînes caractère par caractère sur les $|q|$ premiers caractères de la requête, ce qui lui donne une complexité en temps de $O(|q|)$ par itération. La boucle effectuant exactement m itérations, la complexité totale de `commonPrefix` est donc $\Theta(m \cdot |q|)$.

Nous pouvons à présent rassembler les différents éléments qui composent `acComplete` et leurs complexités en temps:

- `BinarySearchLow` : $O(q \cdot \log N)$, car on effectue une recherche dichotomique sur les N termes.
- `commonPrefix` : $O(m \cdot |q|)$, car on effectue m itérations en appelant `strncmp` à chaque fois.
- `sort` : $\Theta(m \log m)$ ou $\Theta(m^2)$ pour le meilleur ou le pire cas de `QuickSort` et $\Theta(m \log(m))$ pour `MergeSort`.

Afin de déterminer la complexité en temps de l'algorithme `AutoCompleteSort`, nous devons faire la somme de toutes les complexités ci-dessus:

$$O(q \log(N) + O(m \cdot q) + \Theta(m \log(m))$$

Terme	Calcul	Valeur
$\|q\| \cdot \log N$	$5 \cdot \log_2(250000)$	$5 \cdot 18 = 90$
$m \cdot \|q\|$	$100 \cdot 5$	500
$m \log m$	$100 \cdot \log_2(100)$	$100 \cdot 6.6 \approx 660$

Table 2: Analyse numérique des termes de complexité de `acComplete` pour $N = 250000$, $m = 100$, $k = 10$, $|q| = 5$

On observe que $|q| \cdot \log N = 90$ est négligeable devant $m \cdot |q| = 500$ et $m \log m = 660$. Les termes dominants sont donc $m \cdot |q|$ et $m \log m$, ce qui nous donne une complexité en temps finale de :

$$O\big(m \cdot (|q| + \log m)\big)$$

Pour ce qui en est de la complexité en mémoire, `acComplete` initialise un tableau de taille m pour stocker les termes ayant un préfixe commun à la requête. Dès lors, la complexité en mémoire de `acComplete` est $\Theta(m)$.

3.3 Analyse de complexité de AutoCompletePQ.c:

Principe : Comme pour l'approche précédente, la liste des N termes est d'abord triée par ordre lexicographique et une recherche binaire permet d'identifier les m termes dont la requête est un préfixe. Plutôt que de trier l'intégralité des m termes, on utilise une file à priorité de taille limitée à k pour extraire les k termes de poids les plus élevés. Pour chaque terme parmi les m, on l'insère dans la file si elle contient moins de k éléments, ou si son poids est supérieur au terme de poids le plus faible dans la file, auquel cas ce dernier est remplacé. Les k termes sont finalement obtenus par k extractions du maximum de la file.

3.3.1 Analyse de complexité de `acCreate`

Dans notre implémentation, la fonction `acCreate` trie les données par ordre lexicographique à l'aide d'un des deux algorithmes de tri `MergeSort` ou `QuickSort`. Les données étant dans un ordre quelconque, nous ne nous retrouvons pas dans le pire cas de `QuickSort`. Dès lors, la complexité en temps de cette fonction est de l'ordre de:

$$\Theta(N \log N)$$

peu importe l'algorithme utilisé.

La complexité en mémoire de `acCreate` est de $\Theta(1)$ puisqu'il n'y a aucune allocation de mémorie pour stoquer des valeurs.

3.3.2 Analyse de complexité de acComplete:

Avant de pouvoir calculer la complexité de `acComplete`, nous devons d'abord analyser les fonctions de `PQ.c` (la file à priorité).

- `pqInsert` : $O(\log k)$, car la fonction ajoute l'élément à la fin du tas et déplace l'élément vers le haut de l'arbre avec la fonction `shiftUp`. Cette remontée parcourt au plus $\log k$ niveaux.
- `pqReplaceMax` : $O(\log k)$, car la fonction remplace le haut du tas avec l'élément qui est ensuite redescendu vers le bas de l'arbre avec la fonction `shiftDown`. Cette descente parcourt au plus $\log k$ niveaux.
- `pqExtractMax` : $O(k \log k)$, car on extrait au plus k éléments du tas, chaque extraction coûtant $O(\log k)$.

Les fonctions sont de l'ordre de O car elles peuvent s'arrêter à n'importe quel moment.

Nous pouvons à présent rassembler les différents éléments qui composent `acComplete` et leurs complexités en temps:

- `BinarySearchLow`: $O(q \cdot \log N)$, car on effectue une recherche dichotomique sur les N termes.
- `commonPrefix`: $O(m \cdot |q|)$, car on effectue m itérations en appelant `strncmp` à chaque fois.
- `pqInsert` ou `pqReplaceMax`: $O(m \log(k))$ car ces deux fonctions se trouvent dans une boucle qui parcourt les m termes identifiés par `BinarySearch` et parcourt les $\log(k)$ couches qui se trouvent dans le tas.
- `pqExtractMax`: $O(k \log k)$

Afin de déterminer la complexité en temps de l'algorithme `AutoCompletePQ`, nous devons faire la somme de toutes les complexités ci-dessus:

$$O(q \cdot \log N) + O(m \cdot |q|) + O(m \log(k)) + O(k \log k)$$

Nous allons donc utiliser une approche numérique qui nous permettra de d'identifier les termes négligeables:

Terme	**Calcul**	**Valeur**
$\|q\| \cdot \log N$	$5 \cdot \log_2(250000)$	$5 \cdot 18 = 90$
$m \cdot \|q\|$	$100 \cdot 5$	500
$m \log k$	$100 \cdot \log_2(10)$	$100 \cdot 3.3 \approx 330$
$k \log k$	$10 \cdot \log_2(10)$	$10 \cdot 3.3 \approx 33$

Table 3: Analyse numérique des termes de complexité de `acComplete` pour $N = 250000$, $m = 100$, $k = 10$, $|q| = 5$

On observe que $k \log k = 33$ et $|q| \cdot \log N = 90$ sont négligeables devant $m \cdot |q| = 500$ et $m \log k = 330$. Les termes dominants sont donc $m \cdot |q|$ et $m \log k$, ce qui nous donne une complexité en temps finale de :

$$O\big(m \cdot (|q| + \log k)\big)$$

La complexité en espace de `acComplete` est $\Theta(k)$, puisqu'on alloue une file à priorité de capacité k pour stocker les k meilleurs termes.

3.4 Résumé de l'analyse des complexités des 3 algorithmes d'auto-complétions:

Algorithme	**Fonction**	**Complexité en temps**	**Complexité en mémoire**
`AutoCompleteLinear`	`acCreate` `acComplete`	$\Theta(1)$ $\Theta(k)$ meilleur cas, $\Theta(N)$ pire cas	$\Theta(1)$ $\Theta(k)$
`AutoCompleteSort`	`acCreate` `acComplete`	$\Theta(N \log N)$ $O(m \cdot (\|q\| + \log m))$	$\Theta(N)$ $\Theta(m)$
`AutoCompletePQ`	`acCreate` `acComplete`	$\Theta(N \log N)$ $O(m \cdot (\|q\| + \log k))$	$\Theta(1)$ $\Theta(k)$

Table 4: Récapitulatif des complexités en temps et en mémoire des trois approches

4 Comparaison empirique des trois approches d'auto-complétion:

Les figures 1, 2 et 3 présentent les temps moyens d'exécution de `acComplete` en fonction de la longueur de la requête l, pour $k = 20$ et 100 répétitions par valeur de l, sur les trois fichiers de données.

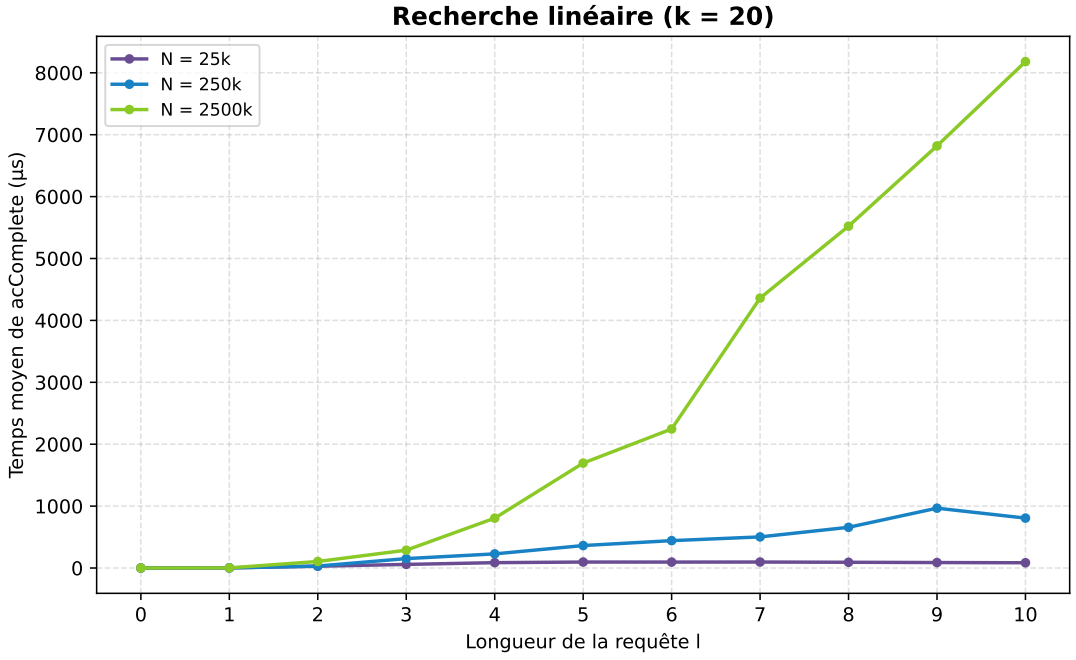


Figure 1: Temps moyen de `acComplete` en fonction de l pour la recherche linéaire.

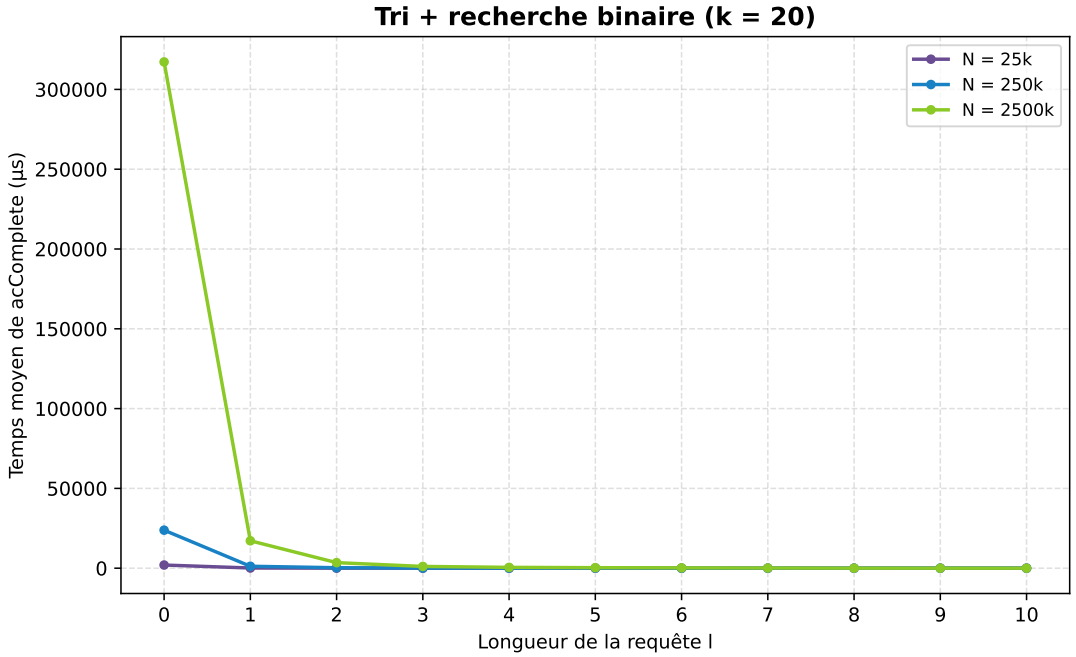


Figure 2: Temps moyen de `acComplete` en fonction de l pour le tri + recherche binaire.

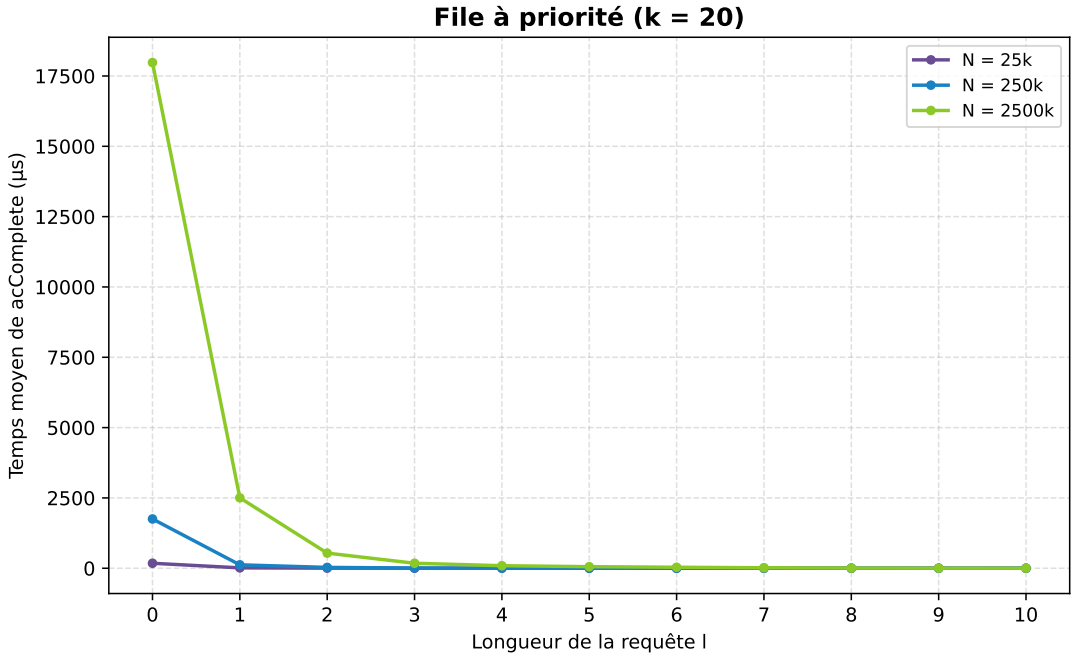


Figure 3: Temps moyen de `acComplete` en fonction de l pour la file à priorité.

5 Discussion des résultats expérimentaux

Nous allons discuter des résultats obtenus dans la section précédente. Pour information, l'algorithme de tri utilisé est `MergeSort`, comme nous l'avons sélectionner à la question 2.

5.1 AutoCompleteLinear

Nous pouvons constater que `AutoCompleteLinear` est très efficace lorsque la taille de la requête est nulle. Ce résultat est logique, puisque l'algorithme s'arrête lorsqu'il trouve k termes compatibles avec la requête. Dans le cas où la requête est nulle, l'algorithme sélectionne les k premiers termes. Plus la taille de la requête augmente, plus l'algorithme tendra à parcourir les N termes pour satisfaire le quota des k termes ayant le même préfixe que la requête. On observe également que l'impact de N est significatif: plus le nombre de termes est grand, plus le temps d'exécution est élevé.

5.2 AutoCompleteSort

Pour `AutoCompleteSort`, on observe un comportement inverse: le temps d'exécution est très élevé pour $l = 0$ puis diminue rapidement à mesure que l augmente. Cela s'explique par le fait qu'à $l = 0$, la requête est vide et donc $m = N$ termes correspondent au préfixe. L'algorithme doit alors trier l'intégralité des N termes par poids. Plus l augmente, plus m diminue, ce qui réduit le coût du tri et donc le temps d'exécution.

5.3 AutoCompletePQ

`AutoCompletePQ` présente un comportement similaire que `AutoCompleteSort`, avec des temps très élevés à $l = 0$ qui diminuent rapidement. Cependant, les temps sont nettement inférieurs à ceux de `AutoCompleteSort`, car la file à priorité évite de trier l'intégralité des m termes. Elle maintient uniquement les k meilleurs termes.

6 Conclusion

En conclusion, l'algorithme d'auto-complétion le plus adapté est `AutoCompletePQ`. En effet, pour des requêtes courtes (l petit), m sera grand mais la file à priorité évite de trier l'intégralité des m termes, ce qui la rend plus rapide que `AutoCompleteSort`. Ce qui se traduit en une meilleure réactivité en début de frappe. De plus, pour des requêtes plus longues, les trois algorithmes sont comparables. En pratique, les utilisateurs tapent progressivement, leurs requêtes devraient être, pour la plupart, très courtes.

7 Remarques:

Pour les mesures empiriques de `QuickSort`(table 1), nous pouvons constater que les performances sont mauvaises lorsqu'on trie par poids. Cela est dû à un mauvais choix de pivot dans mon algorithme. En effet, avant de commencer le projet, je n'avais pas remarqué que les données(`actors_25k.txt, actors_250k.txt` et `actors_2500k.txt`) étaient triées par ordre de poids. Mon implémentation prend donc à chaque fois le dernier pivot, ce qui nous mène au pire cas d'utilisation de `QuickSort`.